

APS Database (aps-db)

PostgreSQL database for the Apartment Booking System (APS). This repository contains only the database layer: schema, seed data, and Docker setup.

Overview

This repository provides:

- PostgreSQL schema (SQL migrations)
- Optional seed data for testing
- Docker Compose configuration
- Basic database management scripts

The database is intended to be reusable by multiple backends (e.g., REST or GraphQL), but this repository focuses **only on the database itself**.

Schema

Tables

Table	Description
users	User accounts and roles
facilities	Bookable facilities
bookings	Facility reservations

Design Notes

- Foreign key constraints are enforced
 - Business rules are implemented using CHECK constraints
 - Indexes are added for common query patterns
 - Booking conflicts are prevented using a partial unique index
 - Timestamps are automatically maintained
-

Quick Start (Windows)

Prerequisites

- Docker Desktop for Windows
 - Docker Compose (included with Docker Desktop)
 - PostgreSQL client (optional, e.g. psql via pgAdmin)
-

1. Environment Setup

```
copy .env.example .env
```

Edit `.env` if needed. Default values work for local development.

2. Start the Database

```
docker-compose up -d
```

Check container status:

```
docker-compose ps
```

On first startup, PostgreSQL will automatically execute migration files mounted into the container.

3. Seed Data (Optional)

If you want test data:

```
docker-compose exec postgres psql -U apsaadmin -d apartment_booking -f /docker-  
entrypoint-initdb.d/seeds/001_users.sql  
docker-compose exec postgres psql -U apsaadmin -d apartment_booking -f /docker-  
entrypoint-initdb.d/seeds/002_facilities.sql  
docker-compose exec postgres psql -U apsaadmin -d apartment_booking -f /docker-  
entrypoint-initdb.d/seeds/003_bookings.sql
```

Note:

- Seed data should typically be applied **once**
 - Do not reseed production databases
-

4. Verify Database

```
docker-compose exec postgres psql -U apsaadmin -d apartment_booking
```

Inside `psql`:

```
\dt  
SELECT COUNT(*) FROM users;
```

```
SELECT COUNT(*) FROM facilities;  
SELECT COUNT(*) FROM bookings;
```

Exit:

```
\q
```

Accessing the Database

Option 1: psql via Docker

```
docker-compose exec postgres psql -U apsdadmin -d apartment_booking
```

Option 2: pgAdmin (Web UI)

1. Open: <http://localhost:5050>
2. Login:
 - Email: `admin@aps.local`
 - Password: `admin123`
3. Register Server:
 - Host name: `postgres`
 - Port: `5432`
 - Database: `apartment_booking`
 - Username: `apsadmin`
 - Password: value from `.env`

Option 3: External Database Tools

Connection details:

```
Host: localhost  
Port: 5432  
Database: apartment_booking  
Username: apsdadmin  
Password: (from .env)
```

Works with tools like pgAdmin Desktop, DBeaver, or DataGrip.

Common Queries

User Role Distribution

```
SELECT role, is_verified_tenant, COUNT(*)
FROM users
GROUP BY role, is_verified_tenant;
```

Booking Status Distribution

```
SELECT status, COUNT(*)
FROM bookings
GROUP BY status;
```

Today's Bookings

```
SELECT
  b.id,
  u.full_name,
  f.name AS facility,
  b.start_time,
  b.end_time,
  b.status
FROM bookings b
JOIN users u ON b.user_id = u.id
JOIN facilities f ON b.facility_id = f.id
WHERE b.booking_date = CURRENT_DATE
ORDER BY b.start_time;
```

Detect Booking Conflicts (Expected Result: 0 rows)

```
SELECT
  b1.facility_id,
  b1.booking_date,
  b1.start_time,
  b1.end_time,
  b2.start_time,
  b2.end_time
FROM bookings b1
JOIN bookings b2
  ON b1.facility_id = b2.facility_id
  AND b1.booking_date = b2.booking_date
```

```
AND b1.id <> b2.id
WHERE
  b1.status NOT IN ('cancelled', 'expired')
AND b2.status NOT IN ('cancelled', 'expired')
AND (b1.start_time, b1.end_time)
  OVERLAPS (b2.start_time, b2.end_time);
```

Directory Structure

```
aps-db/
├── migrations/           # Database schema (run in order)
│   ├── 001_create_users_table.sql
│   ├── 002_create_facilities_table.sql
│   ├── 003_create_bookings_table.sql
│   └── 004_create_indexes.sql
├── seeds/               # Optional test data
│   ├── 001_users.sql
│   ├── 002_facilities.sql
│   └── 003_bookings.sql
├── docker-compose.yml   # PostgreSQL + pgAdmin
├── .env.example         # Environment template
└── README.md            # Documentation
```

Resetting the Database (Destructive)

```
docker-compose down -v
docker-compose up -d
```

This removes all data volumes and recreates the database from scratch.

Troubleshooting

Database Container Not Running

```
docker-compose logs postgres
```

Connection Issues

```
docker-compose ps
```

Ensure port 5432 is not used by another PostgreSQL installation on Windows.

Migration Issues

```
docker-compose exec postgres psql -U apsaadmin -d apartment_booking -c "\dt"
```

If needed, reset the database using `docker-compose down -v`.

License

MIT License
